STRATOS · Game Theory course project

Complete Product and Technical Viva Handbook

Strategic Behavior in University Course / Project Team Formation
For all four team members: read before the physical viva

Document	STRATOS Complete Product and Technical Viva Handbook, version 1.0
Generated	2026-10-26
Built from commit	cdd6ec01fb3d7689d591c7a81152c021a5149992 (branch release/final-product); the PDFs a
Tests recorded on	34e2007: 158 passed, 0 failed, 0 skipped
Working tree at build	clean
Team	Aryan Rajguru, Harsh Kumar Singh, Ayush, Kashish
Status	Pending verification items are listed in section 0; read them first
Data	All student data and weights are fictional and synthetic

Contents

How to use this handbook . . . 4

0. Status at a glance: what is verified, and what is still pending . . . 4

1. Project overview . . . 5

1.1 Title and context . . . 5

1.2 Problem statement and motivation . . . 5

1.3 Why Game Theory is the right tool . . . 5

1.4 What STRATOS does . . . 5

1.5 Intended users, assumptions, scope and limitations . . . 6

1.6 Implemented versus only proposed . . . 6

2. Complete architecture . . . 6

2.1 Files and responsibilities . . . 7

2.2 How data moves from the UI to the engine and back . . . 8

2.3 How the application starts . . . 8

2.4 Dependencies and what each is for . . . 8

3. The mathematical model, from first principles . . . 9

3.1 Players, strategies, assignments . . . 9

3.2 Parameters: preference, compatibility, crowding strength . . . 9

3.3 Crowding cost . . . 10

3.4 Utility . . . 10

3.5 Social welfare . . . 10

3.6 Unilateral deviation and best response . . . 11

3.7 Pure Nash equilibrium . . . 11

3.8 The potential function and why the game always has an equilibrium . . . 11

3.9 Best-response dynamics and its stopping conditions . . . 12

3.10 Pareto optimality, welfare gap, price of stability and price of anarchy . . . 13

4. Worked example by hand (and confirmed by the engine) . . . 13

5. Exhaustive enumeration and results . . . 14

5.1 How the enumeration works . . . 14

5.2 Results for the default scenario . . . 15

5.3 The 23 equilibria, ordered by welfare . . . 16

5.4 Where best-response dynamics ends . . . 16

5.5 How the results were independently checked, and what is still missing . . . 17

6. The frontend and the live demonstration . . . 17

6.1 Every control, what it changes, what it shows . . . 18

6.2 The four tabs, with real screenshots . . . 18

6.3 How to demonstrate stability versus maximum welfare . . . 22

7. Testing and validation 24

7.1 What each test group establishes, and what it does not 25

7.2 Real engine versus mock engine 25

7.3 Known limitations and defects (stated honestly) 25

8. Dependencies, installation and execution (Windows, verified) 25

Roles of the four packages 26

If something goes wrong 26

9. Likely viva questions and model answers 27

9.1 Project motivation and scope 27

9.2 Game Theory fundamentals 28

9.3 Mathematical model and equations 29

9.4 Nash equilibrium and potential games 30

9.5 Algorithms and implementation 30

9.6 Architecture and frontend 31

9.7 Testing and independent validation 32

9.8 Results and interpretation 33

9.9 Limitations, trade-offs and future improvements 33

9.10 Individual contribution and teamwork 34

10. Four-member contribution and ownership map 34

Who can be asked about what 36

How to use this handbook

Read sections 0 to 5 once, front to back: they give you the whole product and the mathematics. Use section 6 together with the running app. Section 9 is the question bank for rehearsal (cover the answers and speak yours aloud). Section 10 tells you what you may truthfully say about who did what. The companion document, **STRATOS_Viva_Speaking_Plan_and_Demo.pdf**, tells each person what to say and do on the day.

> **Ground rules for the viva**
>
> 1. Say **"stable against unilateral deviations"**, never "optimal".
> 2. Say the data is **synthetic**.
> 3. Claim only what you did. For everything else say "I understand it and can explain it".
> 4. If you do not know, say how you would find out (for example: "I would check `docs/MODEL.md`" or "I would run the engine report").

0. Status at a glance: what is verified, and what is still pending

This page is deliberately blunt. Everything marked **verified** was actually run on the commit named on the cover. Everything marked **pending** has not been done, or its evidence is not in the repository, and **must not be claimed** at the viva.

Item	Status	Evidence (where to look)
Automated test suite	**verified**: 158 passed, 0 failed, 0 skipped	`docs/results/pytest_output.txt` (run on commit 34e2007, clean tree, 2026-10-26 02:11:39 +0530)
Engine results recomputed independently of the engine code	**verified** (by a cross-check script, see note below)	`scripts/independent_crosscheck.py --compare` → IDENTICAL; `docs/results/independent_crosscheck_output.txt`
Application runs with the real engine in a real browser	**verified** (automated Chrome run, not a manual one)	`screenshots/INDEX.md`, `screenshots/walkthrough_log.json`: 17 steps, on-screen values matched the engine, 0 console errors
Manual walkthrough on the presentation machine	**pending**	Harsh (frontend owner) + whoever demos; use the checklist in the speaking plan
Formal independent checker and Excel oracle (plan tasks Q4, Q5, O-01..O-04)	**pending / not in repository**	Expected at `tests/oracle/` (Ayush). Not present when this handbook was built.
Official project title, group row, faculty rules (`Groups.xlsx`, handout)	**pending**	Not available to the engineer; plan task Q1 (Ayush)
README.md	**prepared, needs owner review**	`README.md` was written from the frontend draft and the engine docs; Ayush owns the final README
Technical report, slides, test log, demo script, `docs/VIVA.md`	**pending / not in repository**	Plan tasks Q6-Q13 (Ayush)
Screenshots	**verified, genuine**	`screenshots/` (17 PNGs, captured from the running app)

Note on "independent"

The cross-check recomputes the model from `docs/MODEL.md` with different code (it never imports the engine for its own numbers, and it uses exact fractions). But it was prepared for Aryan, the same person who owns the engine and the specification, so it is **independent of the engine's code, not independent of its author**. It supports the numbers; it does not replace the team's formal independent verification by a second person (Ayush's oracle), which is still to be done.

1. Project overview

1.1 Title and context

Working title (from the task brief): *Strategic Behavior in University Course / Project Team Formation*. STRATOS is our Game Theory course project. The software, the mathematics and the evidence are all in one repository: `https://github.com/AryanRajguru22/STRATOS`.

PENDING: official title and faculty requirements

The exact assigned title, the group row in `Groups.xlsx` and any faculty-specific rules have **not been verified** against the official files (they were not available to the engineer; the team plan assigns this check to Ayush as task Q1). Until Ayush confirms, say "our topic is strategic behaviour in course/project team formation" and do **not** quote an official title or rubric.

1.2 Problem statement and motivation

A class of students must end up on project teams. Each student cares about three things: how much they like the project, how well they work with the particular classmates who end up on the same project, and how crowded the team is. If students could choose for themselves, what each one gets depends on what **everyone else** chooses, which is exactly the situation Game Theory studies.

A central planner could simply maximise the total satisfaction of the class. But an assignment that is best on paper is not necessarily one that people will stay in: if some student could do better by switching alone, the assignment is fragile. STRATOS therefore asks three questions:

1. Which assignments are **stable**, meaning no single student gains by switching on their own?
2. How do the stable assignments compare with the assignment that maximises total welfare?
3. If students repeatedly switch to their best project, do they settle down, and where?

1.3 Why Game Theory is the right tool

- **Strategic interdependence.** A student's utility depends on teammates' choices and on team size, so decisions cannot be analysed one student at a time.
- **A solution concept that fits the question.** A pure Nash equilibrium is exactly "nobody wants to move alone".
- **Structure we can prove things about.** With symmetric compatibility and a crowding cost that depends only on team size, the game is an exact potential game: a pure equilibrium always exists and best-response dynamics always terminates (section 3.8).
- **Measures of efficiency.** Welfare gap, price of stability and price of anarchy quantify what stability costs compared with the best assignment.

1.4 What STRATOS does

- Models 8 fictional students choosing among 3 projects, with preference, teammate compatibility and a convex crowding cost.
- Provides an engine (`game_engine.py`) that computes utilities, welfare and the potential, finds profitable deviations, checks Nash equilibrium, runs best-response dynamics and enumerates every assignment.

- Provides a Streamlit app with four tabs (Scenario, Assignment, Equilibrium, Simulation) so the model can be explored and demonstrated live.
- Provides reproducible evidence: a command-line report, a test suite, saved results with git provenance, and an independent cross-check.

1.5 Intended users, assumptions, scope and limitations

Intended users: the project team, the faculty evaluators, and anyone who wants to explore how strategic choice shapes team formation in a small class. STRATOS is a **demonstration and analysis tool, not a deployed allocation system**.

Assumption / limitation	What it means for the claims we can make
All student data and weights are fictional and synthetic	Results show how the model behaves, not what real students would do. The data was chosen so that strategic behaviour is visible.
Complete information, static game	Every student knows every parameter and chooses simultaneously. Best-response dynamics is a simulated adjustment process, not a repeated game.
Only unilateral deviations	A stable assignment can still be improved by two or more students moving together; coalitions are not analysed.
Compatibility is additive and pairwise, and symmetric	This is what makes the potential argument work. An asymmetric or non-pairwise bonus could break it.
Soft crowding instead of a hard capacity	Teams can be any size; large teams are simply costly. Empty teams are allowed.
Exhaustive enumeration is exponential	Capped at 100,000 assignments (about 10 students with 3 projects); the default 8 x 3 case takes about 0.1207 s.
Ties	A tie is not a profitable deviation, so a tied assignment can be an equilibrium (weak definition).

1.6 Implemented versus only proposed

Implemented and evidenced	Not implemented (future work / out of scope)
• Formal model and proof (`docs/MODEL.md`) • Engine with validation, 16 public functions and a CLI report • Four-tab Streamlit app using the real engine • Test suite and saved results with git provenance • Independent cross-check and real-browser screenshots • README and this documentation	• An **Analysis tab** showing equilibria / price of anarchy in the UI (the numbers exist in the CLI report only) • Coalition-proof equilibria; hard team capacity; incomplete information; mixed strategies • Real student data • Heuristics for large classes (enumeration is exponential) • Excel oracle and formal second-person verification (pending) • Report and slides (pending, not in repo)

2. Complete architecture

Everything the user sees is produced by a thin UI layer that calls a self-contained engine. The engine knows nothing about Streamlit, NumPy or pandas, which is why it can be tested and run from the command line on its own.

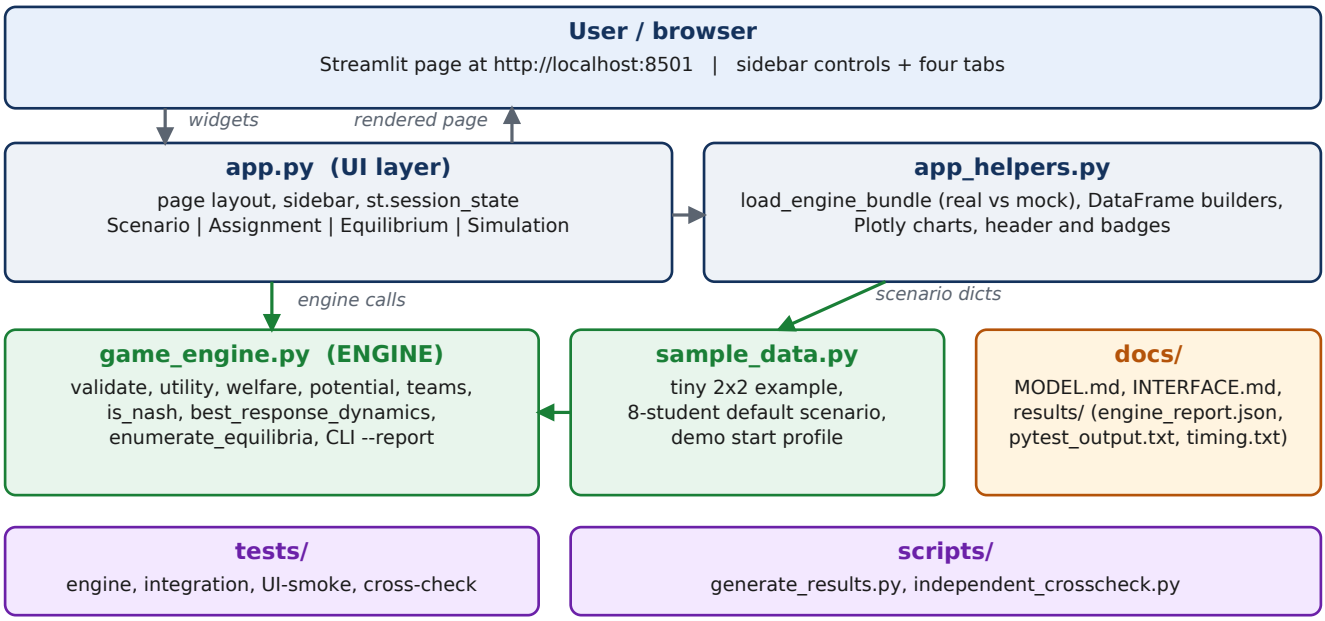


Figure 1. Architecture. Green = the engine and its data (Aryan), blue/grey = UI and helpers (Harsh), amber = documentation and saved evidence, purple = tests and scripts.

2.1 Files and responsibilities

File	Responsibility	Main author (from git history / plan)
`app.py`	Streamlit entry point: page config, sidebar (scenario preset, update order, dynamics seed, max sweeps), session state, the four tabs	Harsh
`app_helpers.py`	`load_engine_bundle()` (real engine, or a labelled mock, or a loud error), DataFrame builders, Plotly chart builders, header banner, `count_students_with_deviations`	Harsh
`game_engine.py`	The engine: validation, utility, welfare, potential, teams, best response, profitable deviations, `is_nash`, dynamics, enumeration, generic Nash finder, `build_report`, CLI	Aryan
`sample_data.py`	Synthetic data: 2x2 hand example, 8-student default scenario, demo start profile, synthetic-data notice	Aryan
`docs/MODEL.md`, `docs/INTERFACE.md`	The formal model with proof; the frozen engine ↔ UI contract (signatures, return keys, errors)	Aryan
`tests/test_game_engine.py`, `test_integration.py`	Engine unit/property tests; engine ↔ UI contract and saved-results checks	Aryan
`tests/test_ui_smoke.py`	AppTest smoke tests, engine-bundle tests, deviation-count regression	Harsh
`scripts/generate_results.py`	Regenerates `docs/results/` from real commands, with git provenance	Aryan
`scripts/independent_crosscheck.py`, `tests/test_independent_crosscheck.py`	Independent recomputation of the results, diffed against the engine	Aryan (prepared with Claude's help)
`screenshots/`	Genuine screenshots of the running app + index + machine-readable log	Aryan's tooling; Ayush owns the screenshots folder in the plan
`README.md`, `requirements.txt`, `requirements-docs.txt`, `pytest.ini`	Setup and reproducibility	Aryan (README to be reviewed by Ayush)

2.2 How data moves from the UI to the engine and back

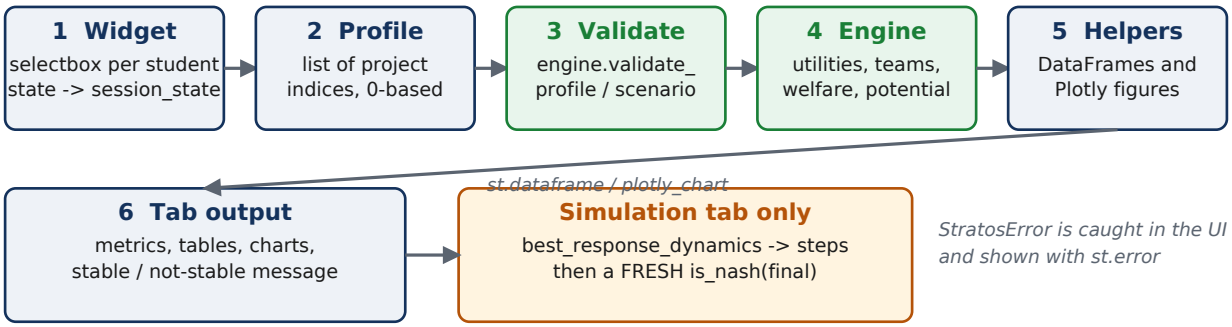


Figure 2. Data flow for the Assignment tab. The same pattern (state → validate → engine → helpers → widgets) is used by the other tabs.

1. The user picks a project for each student in a selectbox. The choices are stored in `st.session_state.profile`, a list with one project index per student (0-based).
2. Before anything is computed the app calls `engine.validate_profile(profile, scenario)`; a `StratosError` is shown with `st.error` instead of crashing.
3. The Assignment tab calls `engine.teams`, `engine.utilities`, `engine.social_welfare` and `engine.potential` with the **same** scenario object.
4. `app_helpers` turns the engine's plain lists and dicts into pandas DataFrames and Plotly figures.
5. The Equilibrium tab calls `engine.is_nash`. The Simulation tab calls `engine.best_response_dynamics`, then **re-checks** the final profile with a fresh `is_nash` call rather than trusting the `converged` flag.

2.3 How the application starts

```
streamlit run app.py
  app.py                 st.set_page_config(...) ; helpers.apply_custom_styles()
  app_helpers.load_engine_bundle()
      real engine importable?  -> EngineBundle(engine=game_engine, sample_data=sample_data, using_mock=False)
      modules missing?          -> labelled mock (or EngineUnavailableError if STRATOS_REQUIRE_REAL_ENGINE=1)
      present but import fails? -> mock + a loud red error box with the traceback (never a silent fallback)
  sidebar                scenario preset, regenerate, update order, dynamics seed, max sweeps
  st.tabs([...])         Scenario | Assignment | Equilibrium | Simulation
```

2.4 Dependencies and what each is for

Package	Declared	Role in STRATOS
Python	3.11 used and tested	Language of every module
Streamlit	`streamlit>=1.51`	Web UI. `width="stretch"` on `st.button`, `st.dataframe` and `st.plotly_chart` requires 1.51 (checked in the 1.50.0 and 1.51.0 wheels)
pandas	`pandas>=2.0`	Tables shown in the UI and the frames built by `app_helpers`
Plotly	`plotly>=5.0`	Heatmaps, bar chart and the welfare/potential line chart (it was missing from `requirements.txt` until commit 859a798)
pytest	`pytest>=7.0`	Test runner; the UI tests use Streamlit's `AppTest`
ReportLab, PyMuPDF	`requirements-docs.txt` only	Used to build these PDFs; **not** needed to run or test the product

The engine itself needs nothing beyond the Python standard library. Tested environment for the evidence in this handbook: Python 3.11.9, Streamlit 1.66.1, pandas 3.1.0, Plotly 7.1.0, pytest 9.1.1 on Windows 11.

Real engine versus mock engine

`app_helpers.py` contains a **mock engine** that mirrors the interface contract. It exists so the frontend could be built before the real engine was ready. With the real engine present it is never used; the app header says "Real engine detected", and the mock badge never appears. Some UI tests (`test_ui_smoke.py`) exercise the **mock** against the hand-solved tiny example, so they prove the mock is faithful, not that the real engine is correct. The real engine's correctness is established by the engine tests, the integration tests and the independent cross-check. Set `STRATOS_REQUIRE_REAL_ENGINE=1` to make any fallback a hard error.

3. The mathematical model, from first principles

Every definition below is the one in `docs/MODEL.md` and in the code. Symbols map to code as: `pref[i][t]` = p(i,t), `w[i][j]` = w(i,j), `gamma` = γ, `utility()`, `social_welfare()`, `potential()`, `is_nash()`.

3.1 Players, strategies, assignments

Players: students 0,…,n−1. The default scenario has n = 8: Asha, Bram, Chen, Dara, Eli, Farah, Goran, Hana. **Strategies:** each student picks exactly one project from 0,…,m−1 (m = 3: C = Campus Navigation App; W = Wellbeing Survey Analysis; E = Event Management Portal). **Assignment (profile):** s = (s_0,…,s_{n-1}), one project per student. Indices are 0-based everywhere in the code.

$$\text{number of assignments} = m^n = 3^8 = 6{,}561$$

Example. The demo start assignment is 0 0 0 1 1 1 2 2, i.e. C: Asha, Bram, Chen; W: Dara, Eli, Farah; E: Goran, Hana.

[model] Q. Why are there exactly 6,561 assignments?

Each of the 8 students independently has 3 choices, and choices are independent, so the count is 3 × 3 × … × 3 (eight times) = 3^8 = 6,561. Empty teams are allowed and there is no capacity limit, so every one of these is a legal assignment.

3.2 Parameters: preference, compatibility, crowding strength

Student	Skills	p(·,C)	p(·,W)	p(·,E)
Asha	Programming, Analysis	8	6	2
Bram	Programming	9	1	5
Chen	Analysis, Communication	3	9	4
Dara	Design, Communication	7	3	8
Eli	Programming, Design	9	2	6
Farah	Analysis	4	8	3
Goran	Communication	2	5	9
Hana	Design, Analysis	6	7	5

Table 1. Default preferences p(i,t) on a 0–10 scale (C = Campus Navigation App, W = Wellbeing Survey Analysis, E = Event Management Portal). Fictional data.

$$w(i,j) = \lambda \cdot |\, S_i \,\Delta\, S_j \,| - \beta, \text{ with } \lambda = 1,\ \beta = 1$$

S_i is student i's skill set; Δ is the symmetric difference (skills held by exactly one of the two). Used only to generate the default w; the engine just reads the matrix w.

Complementary skills help (large symmetric difference); near-identical students help less, and two students with identical skill sets have w = 0 − 1 = −1. w is **symmetric by construction** (w(i,j) = w(j,i)) and has a zero diagonal.

Example. Eli = {Programming, Design} and Dara = {Design, Communication}: the symmetric difference is {Programming, Communication}, size 2, so w(Eli, Dara) = 2 − 1 = 1. Eli and Farah = {Analysis}: symmetric difference has size 3, so w(Eli, Farah) = 3 − 1 = 2. (Both values are what the engine's matrix contains: 1 and 2.)

3.3 Crowding cost

$$c(k) = \gamma \cdot (k-1)^2$$

k = number of students on a project, counting the student themself. $\gamma \geq 0$ is the crowding strength (default $\gamma = 2$).

A student alone on a project pays nothing, c(1) = 0. The cost is **convex**: each extra teammate costs more than the last, which is what stops everyone piling onto their favourite project.

team size k	1	2	3	4	5	6	7	8
c(k), γ = 2	0	2	8	18	32	50	72	98

Example. At the demo start, Eli's team (Dara, Eli, Farah) has k = 3, so each member pays c(3) = 2·(3−1)² = 8. If all 8 students were on one project, each would pay c(8) = 2·7² = 98, which is why the Reset button gives welfare −670.

[model] Q. Why a quadratic crowding cost rather than a hard limit on team size?

A hard cap makes some moves infeasible, which complicates the definition of a deviation and can destroy the potential structure. A convex cost keeps every move feasible, still discourages crowding, and the cost depends only on team size, which is exactly what the potential-game proof needs.

3.4 Utility

$$u_i(s) = p(i, s_i) + \sum_{j \in \text{mates}(i)} w(i,j) - c(k_i)$$

mates(i) = the other students on i's project; k_i = number of students on i's project including i. preference + compatibility with teammates − crowding cost.

Worked example (default scenario, demo start). Eli is on Wellbeing Survey Analysis with Dara and Farah. u = preference 2 + compatibility (1 + 2) − crowding 8 = **−3**.

If Eli alone moved to Event Management Portal (everyone else unchanged) he would join Goran and Hana: u = 6 + (2 + 1) − 8 = **1**, a gain of 4.

[model] Q. Does a student's utility depend on other students' choices?

Yes, in two ways. Who else is on my project changes the compatibility term, and how many are on it changes the crowding cost. My preference term depends only on my own choice. That dependence is what makes this a game rather than 8 separate optimisation problems.

[model] Q. Why does the crowding term count the student themself?

k includes me, so a student alone has k = 1 and pays c(1) = γ·0 = 0. A team of two pays c(2) = γ each. Counting only the others would charge a lone student c(0), which is not defined by the model.

3.5 Social welfare

$$W(s) = \sum_i u_i(s) = \sum_i p(i,s_i) + 2 \sum_{\text{same-team pairs } i<j} w(i,j) - \sum_t n(t)\cdot c(n(t))$$

n(t) = number of students on project t. The factor 2 appears because each compatible pair benefits both members.

Example. At the demo start the utilities are [1, 3, −2, −2, −3, 4, 9, 5], so W = 15. Welfare adds up everyone's utility; it is **not** the same as the potential Φ below, and not the same as stability.

3.6 Unilateral deviation and best response

A **unilateral deviation** is one student i switching to project t while everyone else keeps their choice. Its **gain** is $u_i(t, s_{-i}) - u_i(s)$. It is **profitable** if the gain exceeds $\varepsilon = 10^{-9}$. A student's **best response** is a project that maximises their utility given the others; the engine keeps the current project if it is already a maximiser, otherwise it takes the **lowest-index** maximiser. A student moves only for a strictly positive gain.

Eli's options at the demo start	C	W	E
u_{Eli} if he chooses it	−5	−3	1

Eli's current project is W with utility −3; his best response is E with utility 1, so he has a profitable deviation.

[model] Q. What happens when two projects give a student exactly the same utility?

A tie is not a profitable deviation (the gain is not strictly positive), so the student stays. If the current project is already a maximiser the best response is to stay; if not, the engine picks the lowest-index maximiser so the result is deterministic. Consequently a tied assignment can still be a (weak) equilibrium.

3.7 Pure Nash equilibrium

$$s^* \text{ is a pure Nash equilibrium} \Leftrightarrow \text{for every student } i \text{ and every project } t\text{: } u_i(s^*) \geq u_i(t, s^*_{-i})$$

Equivalently: no student has a profitable unilateral deviation. The test costs n(m−1) utility evaluations, 16 for the default scenario.

Example (not an equilibrium). The demo start has 2 profitable deviations: Dara W→E gains 3 (−2→1); Eli W→E gains 4 (−3→1). So it is **not** stable.

Example (equilibrium). In the tiny example (section 4), [0, 1] is stable: each student would lose by moving alone.

Say it this way

A Nash equilibrium is **stable against unilateral deviations**. Never call it "optimal", "best" or "fair": it may not maximise welfare, may not be Pareto-optimal, need not be unique, and says nothing about two students moving together.

[NE] Q. Define a pure Nash equilibrium in this game in one sentence.

An assignment in which no single student can raise their own utility by switching to a different project while everyone else stays where they are.

3.8 The potential function and why the game always has an equilibrium

$$\Phi(s) = \Sigma_i\, p(i, s_i) + \Sigma_{\text{same-team pairs } i<j}\, w(i,j) - \Sigma_t\, \Sigma_{k=1}^{n(t)}\, c(k)$$

$$u_i(s') - u_i(s) = \Phi(s') - \Phi(s) \text{ for every unilateral deviation of any student } i \text{ from } s \text{ to } s'$$

Exact potential property: a student's gain equals the change in Φ.

Proof idea. Let i move from project a to project t. Only terms involving i and the two teams change. The preference part changes by p(i,t) − p(i,a). The pair part loses i's compatibility with the old

teammates and gains it with the new ones. In the crowding sum, team a shrinks from n(a) to n(a)−1, removing c(n(a)) from what is subtracted, and team t grows to n(t)+1, adding c(n(t)+1). The total change is exactly [p(i,t) + Σ w on t − c(n(t)+1)] − [p(i,a) + Σ w on a − c(n(a))], which is u_i after minus u_i before.

Numerical check. At the demo start Φ = 35. Dara's first move gains 3, and Φ becomes 38: 35 + 3 = 38. ✓

Consequences for this model.

1. Φ takes finitely many values, so some assignment maximises it, and no unilateral move from it can be profitable (it would raise Φ). **A pure equilibrium always exists.**
2. Every profitable move raises Φ by exactly the mover's gain, so Φ strictly increases along best-response dynamics and an assignment can never repeat: **dynamics always terminates.** With integer data each move raises Φ by at least 1, so the number of moves is at most $\Phi_{max} - \Phi_{min}$.
3. The argument needs **symmetric w** and a crowding cost that depends **only on team size**. The code never relies on the proof: it checks the equilibrium condition directly.

> **What the potential does NOT mean**
> Φ is **not** social welfare. A maximiser of Φ is an equilibrium, but an equilibrium need not maximise Φ, and the welfare-maximising assignment need not be an equilibrium. In the default scenario the two welfare-optimal assignments are 0 0 1 0 1 1 2 2 and 0 0 1 0 2 1 2 2; only the second is an equilibrium.

[potential] Q. Why does best-response dynamics have to stop?

Each strictly profitable move increases the potential Φ by exactly the mover's gain. Φ is bounded because there are only 3^8 assignments, so it cannot increase forever, and an assignment cannot repeat (Φ would have to return to an earlier, lower value). Therefore a sweep with no move must eventually occur, and that assignment is an equilibrium.

[potential] Q. What would break the potential-game guarantee?

An asymmetric compatibility matrix (w(i,j) ≠ w(j,i)), or a team bonus that is not a sum over pairs (for example "the team covers every skill"), or a crowding cost that depends on who is on the team rather than how many. In those cases there may be no exact potential and dynamics could cycle. The engine rejects asymmetric w at validation.

3.9 Best-response dynamics and its stopping conditions

```
s = start
for sweep r = 1 .. max_rounds:
    visit order = ascending index (round_robin)  or  a seeded random permutation each sweep (random)
    changed = False
    for i in visit order:
        t* = best response of i given the others      (stay if current project already maximises)
        if u_i(t*, s_-i) > u_i(s) + EPS:  s_i = t*; record the step; changed = True
    if not changed: converged = True; stop
return converged, rounds, steps, final profile, and a FRESH is_nash(final)
```

- **Stops** when a full sweep produces no move (`converged = True`) or when `max_rounds` is used up (`converged = False`, shown in the UI as "Not converged").
- `rounds` counts sweeps **including the last sweep in which nobody moved**.
- `is_nash_final` is recomputed from scratch, so stability is never inferred from `converged` alone.

Observed on the demo start. Round-robin: 3 sweeps, 4 moves, ends at 0 0 1 0 2 1 2 2 with welfare 29. Random order with seed 7: 4 sweeps, 4 moves, ends at 0 0 1 2 2 1 2 0 with welfare 23. Both ends are Nash equilibria, but they are **different** equilibria with different welfare: the outcome depends on the update order.

[dynamics] Q. Does the order in which students update change the answer?

It can. From the same start, round-robin ends at welfare 29 and random order (seed 7) at welfare 23; both are stable. That is why we enumerate all equilibria rather than trusting one run, and why we say dynamics finds *an* equilibrium, not *the best* one.

3.10 Pareto optimality, welfare gap, price of stability and price of anarchy

$$\text{PoS} = W_{\text{opt}} / \max_{\text{equilibria}} W,\ \text{PoA} = W_{\text{opt}} / \min_{\text{equilibria}} W,\ \text{gap} = W_{\text{opt}} - W(s^*)$$

W_{opt} = max over all assignments of W. The ratios are reported only when W_{opt} and every equilibrium welfare are strictly positive; otherwise they are null (None) and the gaps are shown instead.

- **Pareto-optimal** assignment: no other assignment makes every student at least as well off and someone strictly better off.
- **Welfare gap** of an equilibrium: how much total welfare is lost compared with the best assignment (always defined).
- **Price of stability:** how close the *best* equilibrium is to the optimum. **Price of anarchy:** how far the *worst* equilibrium is.

Default scenario. W_{opt} = 29; best equilibrium welfare 29 (gap 0), worst equilibrium welfare 16 (gap 13). PoS = 1.0, PoA = 29/16 = 1.8125. 20 of the 23 equilibria are Pareto-optimal.

Why the ratios can be undefined. With γ = 3 the enumeration still finds 23 equilibria, but some equilibrium welfare is no longer positive, so PoA and PoS are None (None) while the gaps are 0 and 13. The engine returns null rather than a meaningless or negative ratio.

[efficiency] Q. Price of anarchy is 1.8125. What does that number mean?

The best possible total welfare is 29, and the worst stable assignment achieves only 16, and 29 / 16 = 1.8125. So selfish, stable behaviour can cost up to about 45% of the achievable welfare in the worst case (16 is 55% of 29). The price of stability is 1.0 because the best equilibrium reaches the optimum.

4. Worked example by hand (and confirmed by the engine)

The tiny example is small enough to compute everything on paper. Two students, two projects, p = [[5, 1], [1, 5]], w(0,1) = 2, γ = 1, so c(1) = 0 and c(2) = 1. Student 0 likes project 0; student 1 likes project 1; together they are compatible (+2) but a team of two pays a crowding cost of 1 each.

Step 1. Profile [0, 0]: both choose project 0

1. Team sizes: project 0 has k = 2, project 1 is empty. Crowding cost c(2) = 1·(2−1)² = 1 for each of them.
2. u_0 = p(0,0) + w(0,1) − c(2) = 5 + 2 − 1 = **6**.
3. u_1 = p(1,0) + w(1,0) − c(2) = 1 + 2 − 1 = **2**.
4. Welfare W = 6 + 2 = **8**. Potential Φ = (5 + 1) + 2 − [c(1) + c(2)] = 6 + 2 − (0 + 1) = **7**.

Step 2. Is [0, 0] an equilibrium?

1. Student 0 moving to project 1 would be alone there: u_0 = p(0,1) + 0 − c(1) = 1 − 0 = 1 < 6, no gain.
2. Student 1 moving to project 1 would be alone: u_1 = p(1,1) + 0 − c(1) = 5 − 0 = **5** > 2. Gain = 5 − 2 = **3**. A profitable deviation exists, so [0, 0] is **not** a Nash equilibrium.

Step 3. One best-response move

1. Student 1 moves: profile becomes [0, 1].
2. Now each is alone on a project: $u_0 = 5 + 0 - 0 = 5$, $u_1 = 5 + 0 - 0 = 5$. W = **10**.
3. $\Phi(0,1) = (5 + 5) + 0 - [c(1) + c(1)] =$ **10**. Check the potential identity: Φ rose from 7 to 10, i.e. by 3, which is exactly student 1's gain. ✓

Step 4. Is [0, 1] an equilibrium?

1. Student 0 moving to project 1 would share it with student 1: $u_0 = 1 + 2 - c(2) = 1 + 2 - 1 = 2 < 5$.
2. Student 1 moving to project 0 would share it with student 0: $u_1 = 1 + 2 - 1 = 2 < 5$.
3. Nobody gains, so [0, 1] **is** a Nash equilibrium. It also has the highest welfare (10), so here stability and optimality coincide.

profile	utilities	W	Φ	pure Nash?
[0, 0]	[6, 2]	8	7	no
[0, 1]	[5, 5]	10	10	yes
[1, 0]	[1, 1]	2	2	no
[1, 1]	[2, 6]	8	7	no

Table 2. All four profiles of the tiny example, computed by the engine; they match the hand arithmetic above and the table in `docs/INTERFACE.md`*. The app's Tiny hand-verified example preset reproduces this.*

The same ideas on the default scenario: the round-robin trace

move	sweep	student	from → to	gain	welfare after	Φ after
0 (start)	–	–	CCCWWWEE	–	15	35
1	1	Dara	W → E	3	16	38
2	2	Chen	C → W	6	22	44
3	2	Dara	E → C	3	29	47
4	2	Eli	W → E	2	29	49

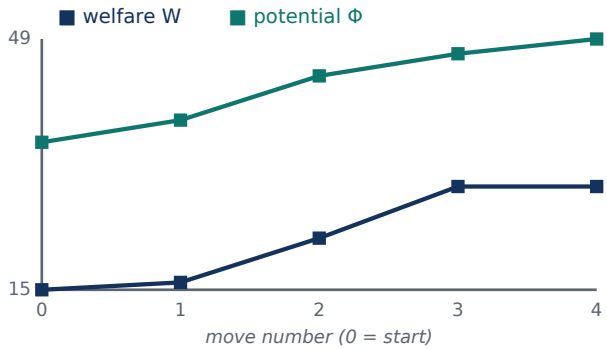


Figure 3. Welfare and potential along the round-robin run from the demo start. Φ rises at every move by exactly the mover's gain (35 → 38 → 44 → 47 → 49); welfare is not guaranteed to rise at every move, though here it does. The run ends at 0 0 1 0 2 1 2 2, an equilibrium with welfare 29.

5. Exhaustive enumeration and results

5.1 How the enumeration works

1. Generate all $3^8 = 6{,}561$ assignments with `itertools.product(range(3), repeat=8)`.
2. For each assignment compute every student's utility, the welfare and the potential.
3. Keep an assignment as an **equilibrium** if no student has a profitable deviation (`_has_deviation`, the same test as `is_nash`).
4. Find the welfare maximiser(s) and the potential maximiser(s).

5. For each equilibrium decide Pareto-optimality by comparing it with **every** other assignment.
6. Guard: if m^n exceeds 100,000 the function raises `StratosError` instead of running for minutes.

Measured on this machine: median 0.1207 s, minimum 0.1194 s for the whole enumeration (5 timed runs, `docs/results/timing.txt`). Timings vary between machines.

5.2 Results for the default scenario

Re-validated, not copied

These five figures were reported earlier. They were **recomputed from scratch** for this document by a fresh engine run and by the independent cross-check (`comparison with game_engine: IDENTICAL`), and agree with the saved report generated on commit `34e2007`.

Quantity	Value	Meaning
Assignments enumerated	6,561	3^8
Pure Nash equilibria	23	assignments with no profitable unilateral deviation
Maximum welfare W_{opt}	29	attained by 2 assignments: 0 0 1 0 1 1 2 2 and 0 0 1 0 2 1 2 2
Is each welfare optimum an equilibrium?	no / yes	only one of the two optima is stable (Eli would gain 2 by moving in the first)
Price of stability	1.0	best equilibrium reaches the optimum
Price of anarchy	1.8125 (= 29/16)	worst equilibrium welfare = 16
Welfare gap, best / worst equilibrium	0 / 13	W_{opt} minus equilibrium welfare
Pareto-optimal equilibria	20 of 23	the other 3 are dominated by some assignment
Potential maximisers	Φ = 49	2 assignments; all are equilibria (as the theory says)

5.3 The 23 equilibria, ordered by welfare

#	assignment	teams (C / W / E)	welfare	Φ	Pareto?
1	0 0 1 0 2 1 2 2	As,Br,Da / Ch,Fa / El,Go,Ha	29	49	yes
2	0 2 1 0 0 1 2 2	As,Da,El / Ch,Fa / Br,Go,Ha	28	48	yes
3	0 0 1 0 2 1 2 1	As,Br,Da / Ch,Fa,Ha / El,Go	27	49	yes
4	1 0 1 0 2 0 2 1	Br,Da,Fa / As,Ch,Ha / El,Go	25	45	yes
5	1 0 1 0 2 1 2 0	Br,Da,Ha / As,Ch,Fa / El,Go	24	46	yes
6	1 2 1 0 0 0 2 2	Da,El,Fa / As,Ch / Br,Go,Ha	24	43	yes
7	2 0 1 2 0 1 2 0	Br,El,Ha / Ch,Fa / As,Da,Go	24	46	yes
8	0 0 1 2 2 1 2 0	As,Br,Ha / Ch,Fa / Da,El,Go	23	47	yes
9	0 2 1 0 2 1 2 0	As,Da,Ha / Ch,Fa / Br,El,Go	22	44	no
10	0 2 1 2 0 1 0 2	As,El,Go / Ch,Fa / Br,Da,Ha	22	42	yes
11	0 2 1 2 0 1 2 0	As,El,Ha / Ch,Fa / Br,Da,Go	22	46	no
12	1 0 1 0 0 1 2 2	Br,Da,El / As,Ch,Fa / Go,Ha	22	46	yes
13	1 2 1 0 0 0 2 1	Da,El,Fa / As,Ch,Ha / Br,Go	22	43	yes
14	0 0 1 2 0 1 2 2	As,Br,El / Ch,Fa / Da,Go,Ha	21	47	yes
15	1 0 1 0 0 2 2 1	Br,Da,El / As,Ch,Ha / Fa,Go	21	44	yes
16	1 0 1 2 0 0 2 1	Br,El,Fa / As,Ch,Ha / Da,Go	21	45	yes
17	0 2 2 2 0 1 1 1	As,El / Fa,Go,Ha / Br,Ch,Da	20	41	yes
18	1 0 1 2 0 1 2 0	Br,El,Ha / As,Ch,Fa / Da,Go	20	46	yes
19	0 0 1 2 0 1 2 1	As,Br,El / Ch,Fa,Ha / Da,Go	19	47	yes
20	0 2 1 2 0 1 1 2	As,El / Ch,Fa,Go / Br,Da,Ha	19	42	yes
21	1 0 0 2 0 1 2 1	Br,Ch,El / As,Fa,Ha / Da,Go	19	43	yes
22	1 0 2 2 0 0 1 1	Br,El,Fa / As,Go,Ha / Ch,Da	18	39	yes
23	0 2 1 2 2 1 0 0	As,Go,Ha / Ch,Fa / Br,Da,El	16	38	no

Table 3. Every pure Nash equilibrium of the default scenario (student names abbreviated to two letters). Source: `enumerate_equilibria`*, identical to* `docs/results/engine_report.json`*.*

5.4 Where best-response dynamics ends

Update order	runs	all converged	all end in Nash	all in enumerated set	moves min / mean / max	max sweeps	distinct endpoints
round_robin	200	True	True	True	0 / 5.44 / 13	5	20
random	200	True	True	True	0 / 5.47 / 11	5	23

From the saved report: 200 seeded random starts per update order (seeds 0–199). Every run converged to a verified equilibrium that is in the enumerated set. The independent cross-check ran round-robin dynamics from **all 6,561** starting assignments: all converged to an equilibrium, using at most 13 moves.

5.5 How the results were independently checked, and what is still missing

Check	Who / what	Result	Limitation
Hand-derived tiny table (4 profiles)	Computed by hand in `docs/INTERFACE.md` section 5; reproduced by engine tests and by the cross-check	match	Tiny scenario only
Recompute everything for the default scenario without engine code	`scripts/independent_crosscheck.py` (exact fractions, own enumeration, own Pareto test, own dynamics)	23 equilibria, W_{opt} 29, PoS 1, PoA 29/16, 20 Pareto: identical to engine	Same author and specification as the engine
Identities on all 6,561 profiles × all deviations	Cross-check: welfare closed form = Σu; exact-potential identity	hold everywhere	Confirms the model is internally consistent, not that the model is right
Generic checker	`pure_nash_equilibria` (engine test: matching pennies returns no pure equilibrium)	pass	Same codebase
Independent second-person re-implementation and Excel oracle	Plan tasks Q4, Q5, O-01..O-04 (Ayush)	**PENDING**: not in the repository	Must be done before it can be cited

6. The frontend and the live demonstration

The app is a thin layer: it holds state, calls the engine and draws the results. It never implements its own model when the real engine is present. Every control below was exercised in a real Chrome session; the screenshots are from that run.

6.1 Every control, what it changes, what it shows

Control (exact label)	Where	What it changes	What to look at
Scenario preset	sidebar	Switches between *Tiny hand-verified example* (2x2), *Default scenario* (8 students, 3 projects, γ = 2) and *Custom mock scenario*. The assignment resets to everyone on project 0.	Scenario tab changes; header still says "Real engine detected".
Number of students / projects, Crowding parameter (gamma), Random seed	sidebar, Custom preset only	Builds a random scenario in the same format (frontend-generated, not the team's dataset).	Useful to show the model at other sizes; still validated by the real engine.
Regenerate scenario	sidebar	Rebuilds the current preset and resets the assignment.	Back to a known state during a demo.
Student order each sweep: round_robin / random	sidebar	Update order used by *Run dynamics*.	Different orders can reach different equilibria (section 3.9).
Dynamics seed (default 7), Max sweeps (default 50)	sidebar	Seed used by the random order; sweep limit.	If the limit is hit the status says *Not converged*.
Per-student selectboxes	Assignment tab	Set one project per student (always a valid profile).	Teams, utilities, welfare and Φ update at once.
Seed box, Random, ↺ Reset	Assignment tab	Random = seeded random assignment (`random_profile`); Reset = everyone on project 0.	Reset on the default scenario gives welfare −670 and many deviations; Random with seed 1 gives [0,2,0,1,0,1,1,1].
▶ Run dynamics	Simulation tab	Runs best-response dynamics from the Assignment tab's assignment (it does not change that assignment).	Sweeps run, Moves made, Status, trace, step slider, welfare/Φ chart.
Inspect profile after move #	Simulation tab	Replays the first k moves.	Teams after k moves; useful for narrating the trace.

6.2 The four tabs, with real screenshots

Scenario tab: the players, the projects and the parameters

Shows the student table with skills, the project list, the crowding parameter and two heatmaps: preferences (0–10) and teammate compatibility w (blue = synergy, red = friction). It is where you say that the data is synthetic (the header carries the engine's synthetic-data notice).

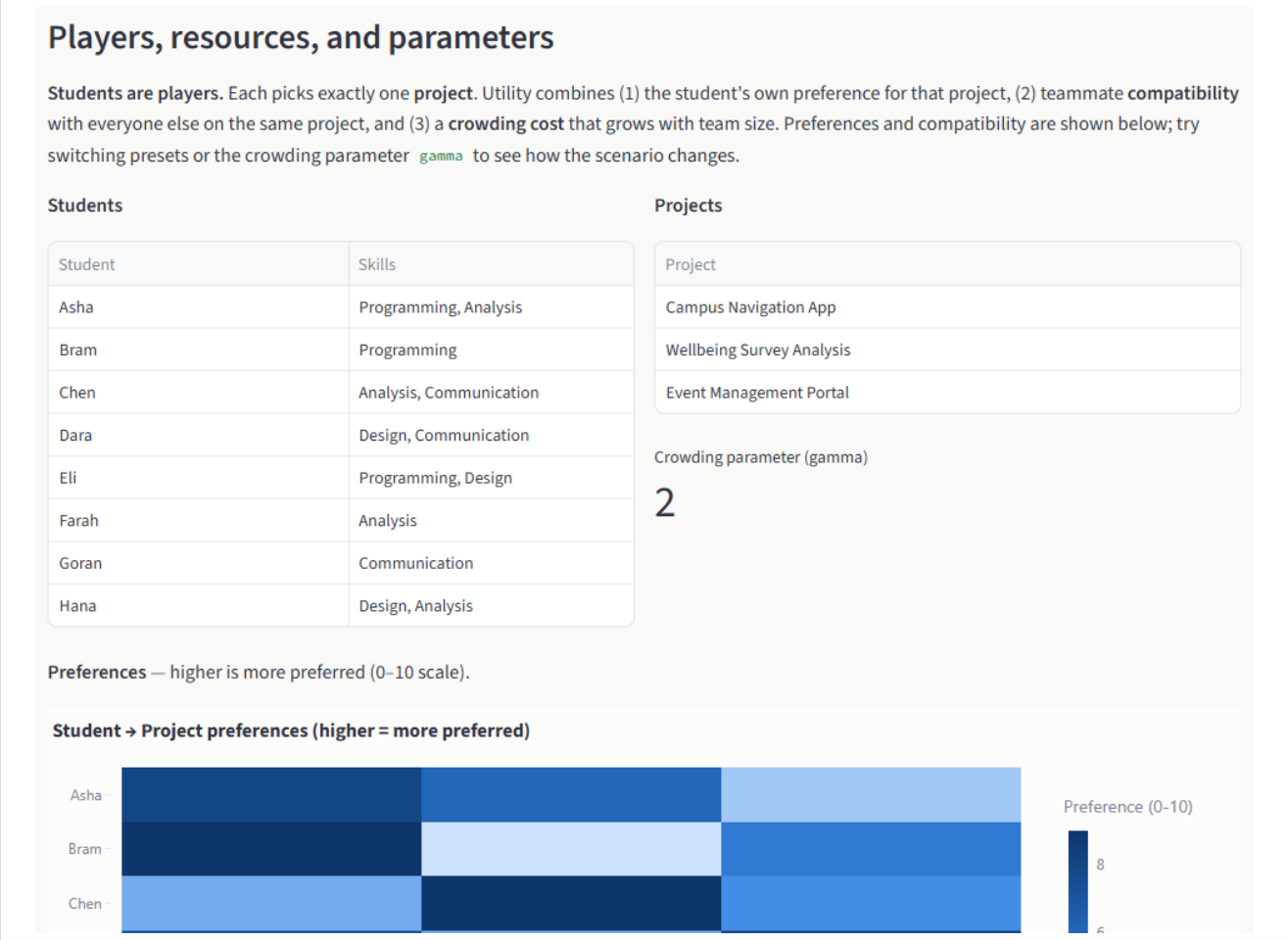


Figure 4. Scenario tab, Default scenario (tables and the start of the preference heatmap). (file `screenshots/S1b_scenario_default_preset.png`*, region of the page; real engine; app code of commit* `c94f918`*; automated Chrome run.)*

Assignment tab: build an assignment and read its numbers

The demo start assignment (Asha→C, Bram→C, Chen→C, Dara→W, Eli→W, Farah→W, Goran→E, Hana→E) has welfare 15 and potential 35. Per-student utilities appear in the table and bar chart.

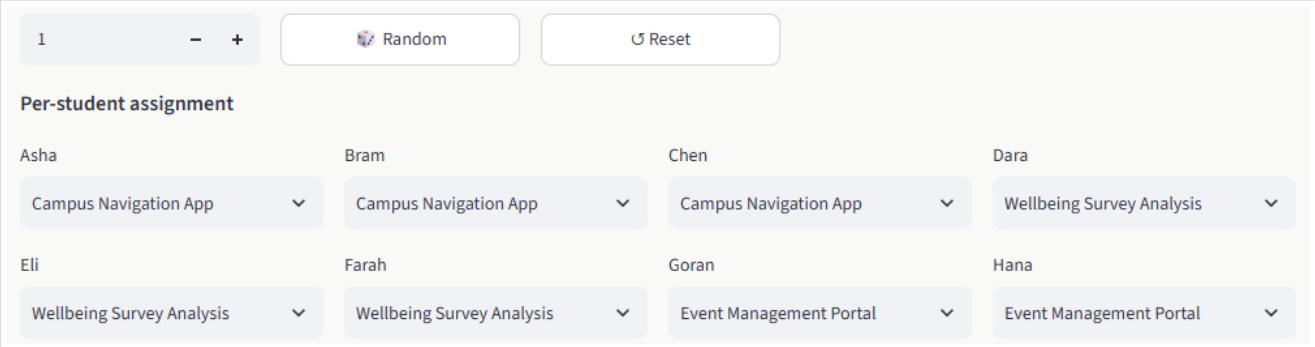


Figure 5a. Assignment tab controls: Random, Reset and one selectbox per student (demo start). (file `screenshots/S2b_assignment_demo_start.png`*, region of the page; real engine; app code of commit* `c94f918`*; automated Chrome run.)*

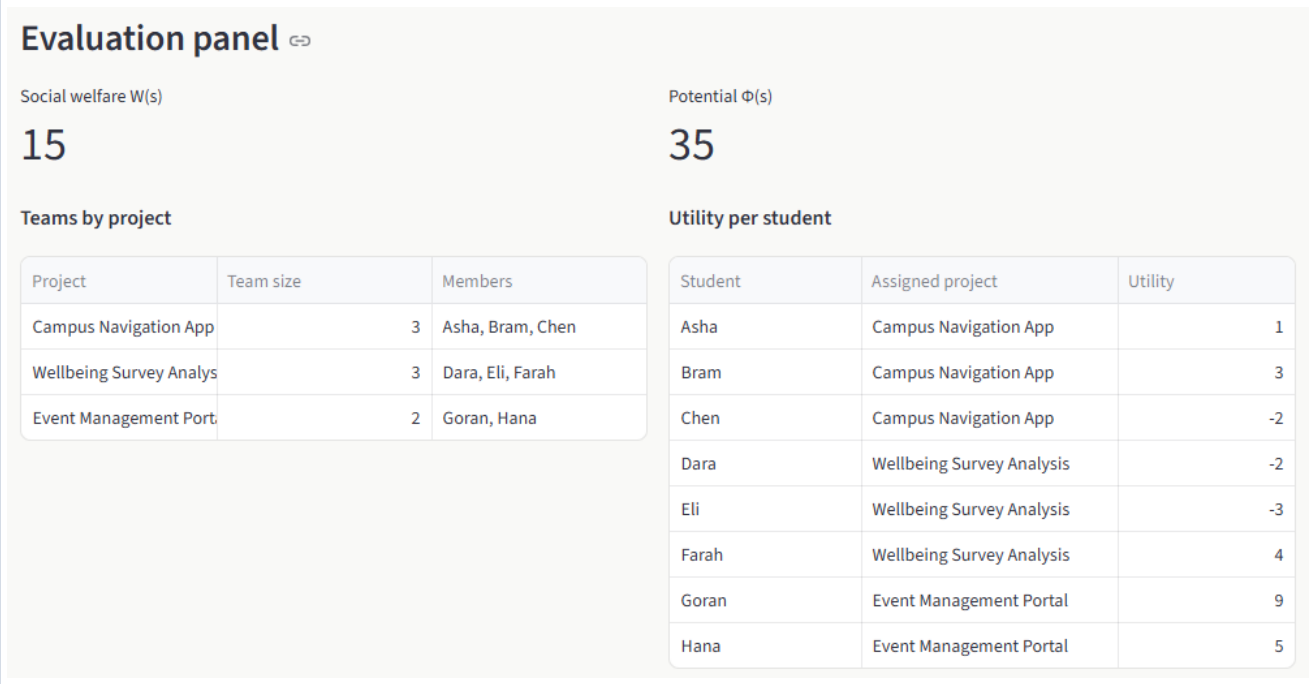


Figure 5b. Evaluation panel at the demo start: welfare 15, potential 35, teams and per-student utilities. (file `screenshots/S2b_assignment_demo_start.png`*, region of the page; real engine; app code of commit* `c94f918`*; automated Chrome run.)*

Equilibrium tab: is this assignment stable?

If not stable, it states how many **distinct students** have a profitable move, gives an example, and lists every profitable deviation with current utility, new utility and gain. (An earlier version counted deviation entries, so all-on-one-project showed "16 student(s)" for 8 students; this was fixed in commit `c94f918` with a regression test.)

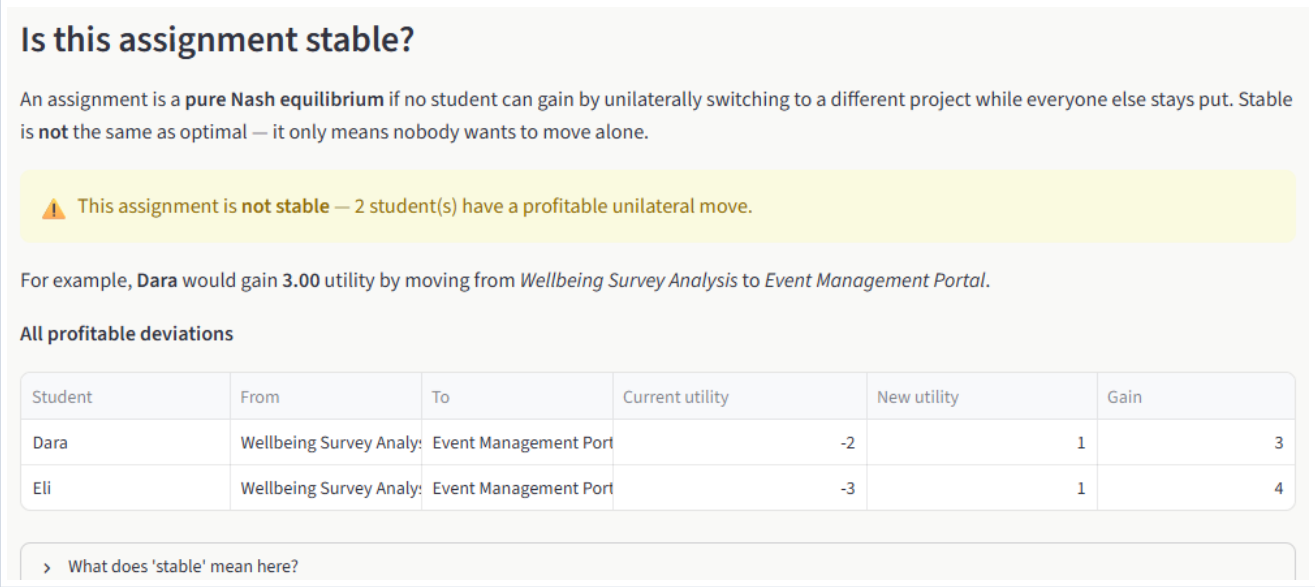


Figure 6. Equilibrium tab at the demo start: 2 students have a profitable move (Dara gains 3, Eli gains 4). (file `screenshots/S3_equilibrium_demo_start_unstable.png`*, region of the page; real engine; app code of commit* `c94f918`*; automated Chrome run.)*

Simulation tab: best-response dynamics

Shows sweeps, moves and status, the move-by-move trace, a slider to inspect any intermediate assignment, and a welfare/potential chart. The final status is **re-verified with a fresh equilibrium check**.

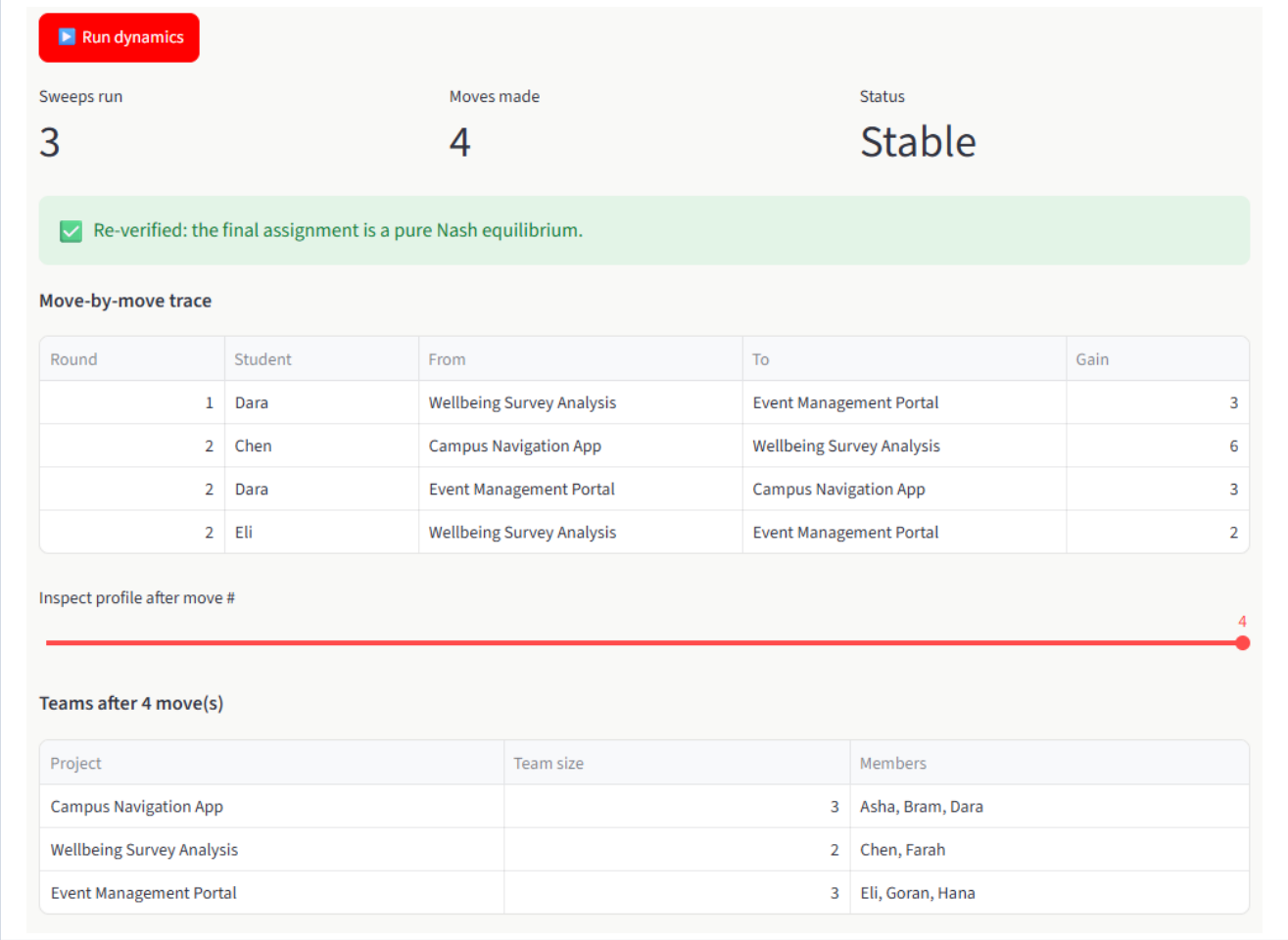


Figure 7. Simulation tab, round_robin from the demo start: 3 sweeps, 4 moves, Stable, ending at welfare 29. (file `screenshots/S4_simulation_round_robin.png`*, region of the page; real engine; app code of commit* `c94f918`*; automated Chrome run.)*

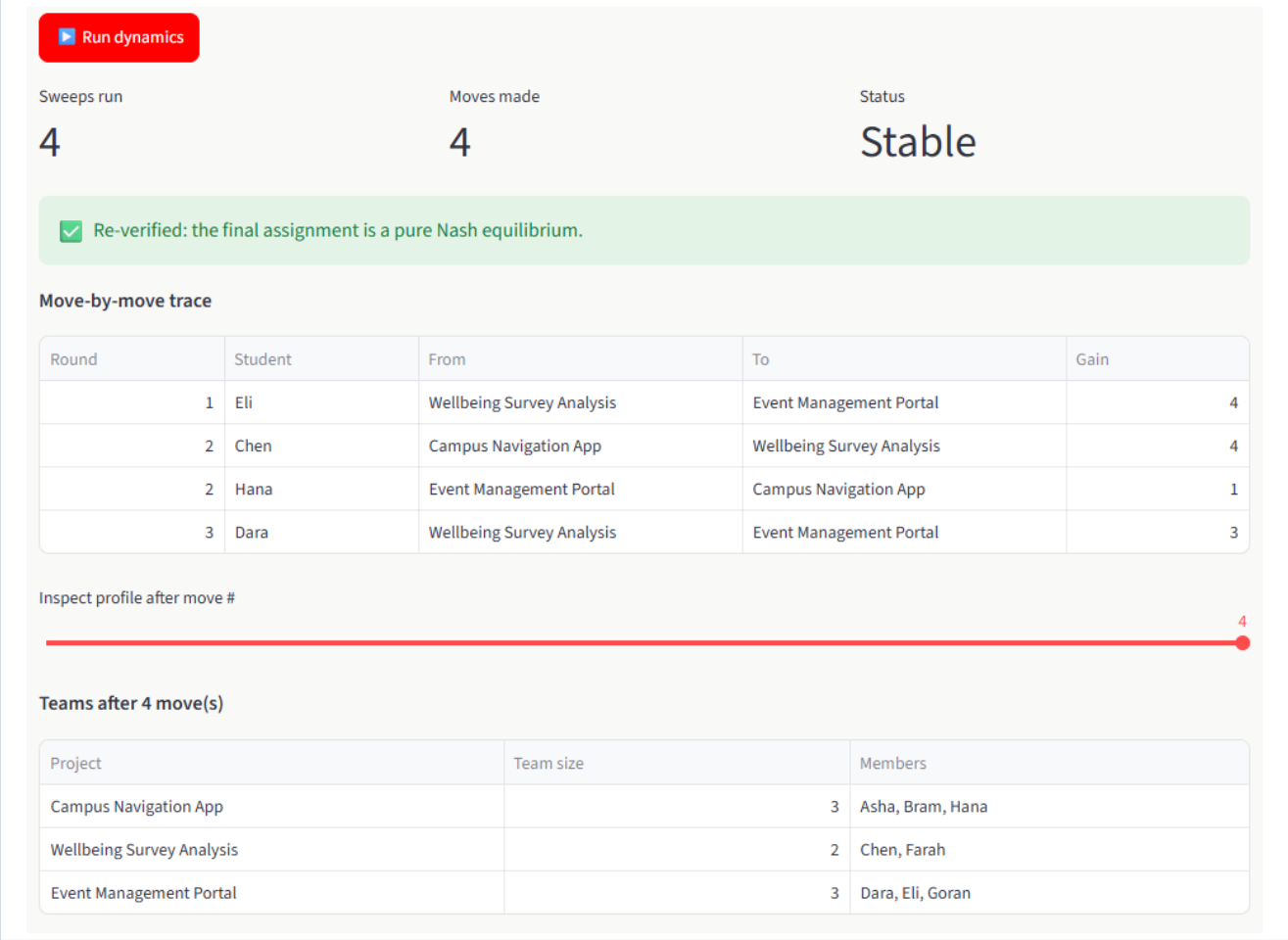


Figure 8. Same start, update order random (seed 7): 4 sweeps, 4 moves, Stable, but a different equilibrium with welfare 23. (file `screenshots/S5_simulation_random_order_seed7.png`*, region of the page; real engine; app code of commit* `c94f918`*; automated Chrome run.)*

6.3 How to demonstrate stability versus maximum welfare

1. On the Assignment tab set the **worst equilibrium** [0, 2, 1, 2, 2, 1, 0, 0] (Asha→C, Bram→E, Chen→W, Dara→E, Eli→E, Farah→W, Goran→C, Hana→C). Welfare **16**, potential 38.
2. Open the Equilibrium tab: it says the assignment is **stable**. Nobody can gain by moving alone, yet 16 is far below the best attainable welfare of 29.
3. Now set [0, 0, 1, 0, 1, 1, 2, 2] (welfare **29**, the maximum). The Equilibrium tab says it is **not stable**: Eli gains 2 by moving. So the best welfare is not even guaranteed to be an equilibrium.
4. Finish by running the dynamics from the demo start with round_robin (ends at the stable welfare-29 assignment) and random (ends at a stable welfare-23 assignment): stable outcomes differ in quality.

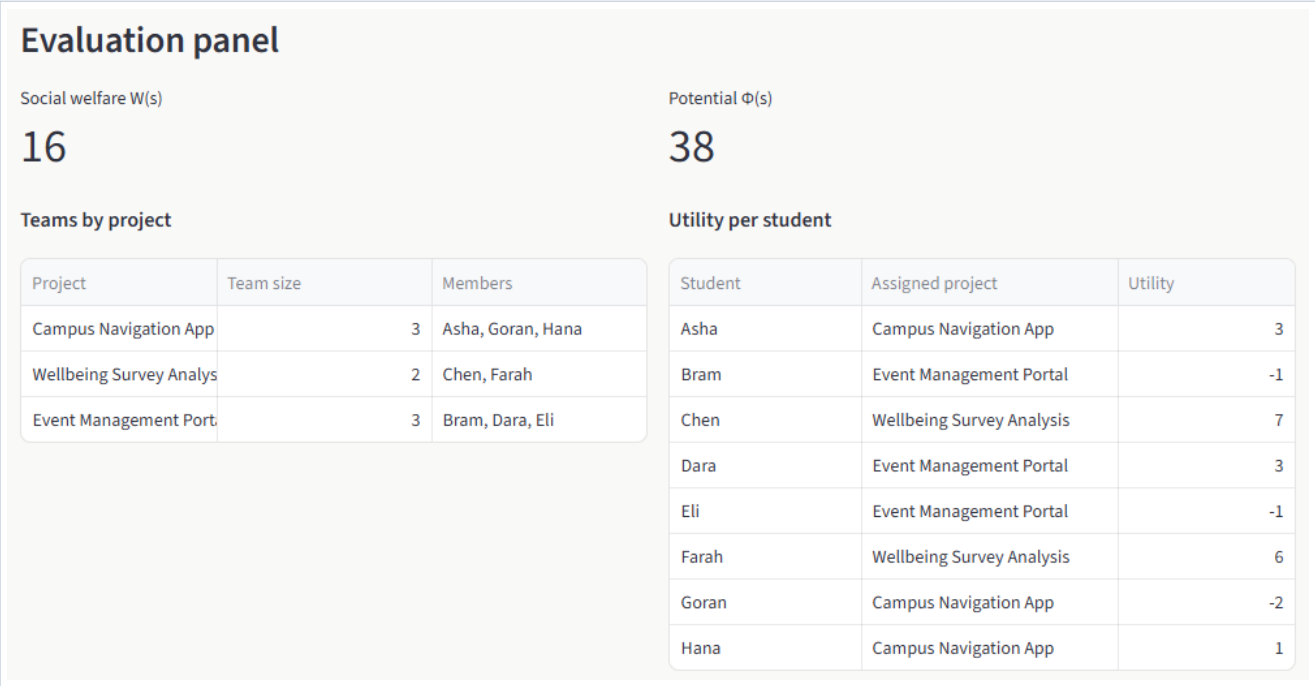


Figure 9. The worst equilibrium on the Assignment tab: welfare 16, potential 38. (file `screenshots/S6_assignment_worst_equilibrium_welfare16.png`*, region of the page; real engine; app code of commit* `c94f918`*; automated Chrome run.)*

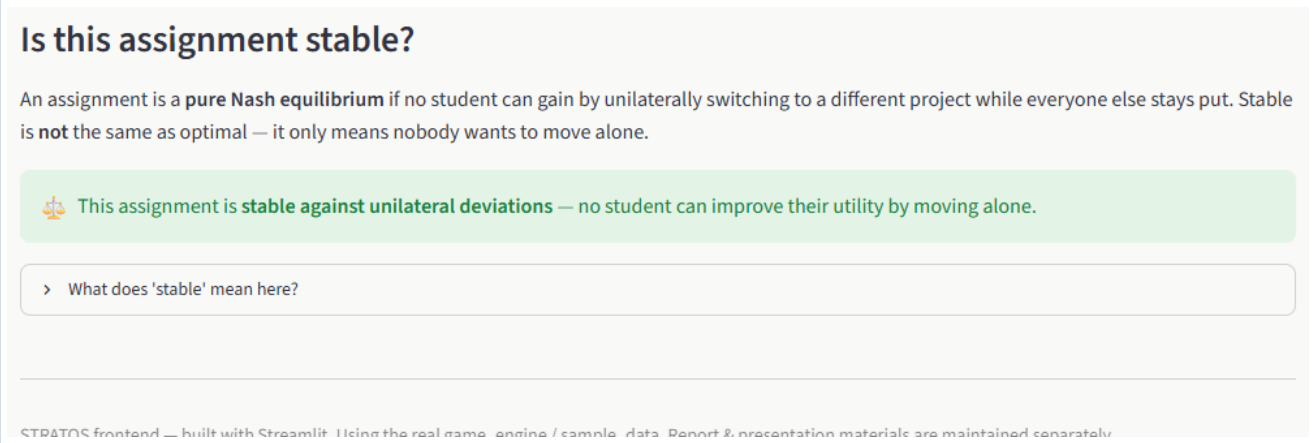


Figure 10. The same assignment on the Equilibrium tab: stable. (file `screenshots/S7_equilibrium_worst_equilibrium_stable.png`*, region of the page; real engine; app code of commit* `c94f918`*; automated Chrome run.)*

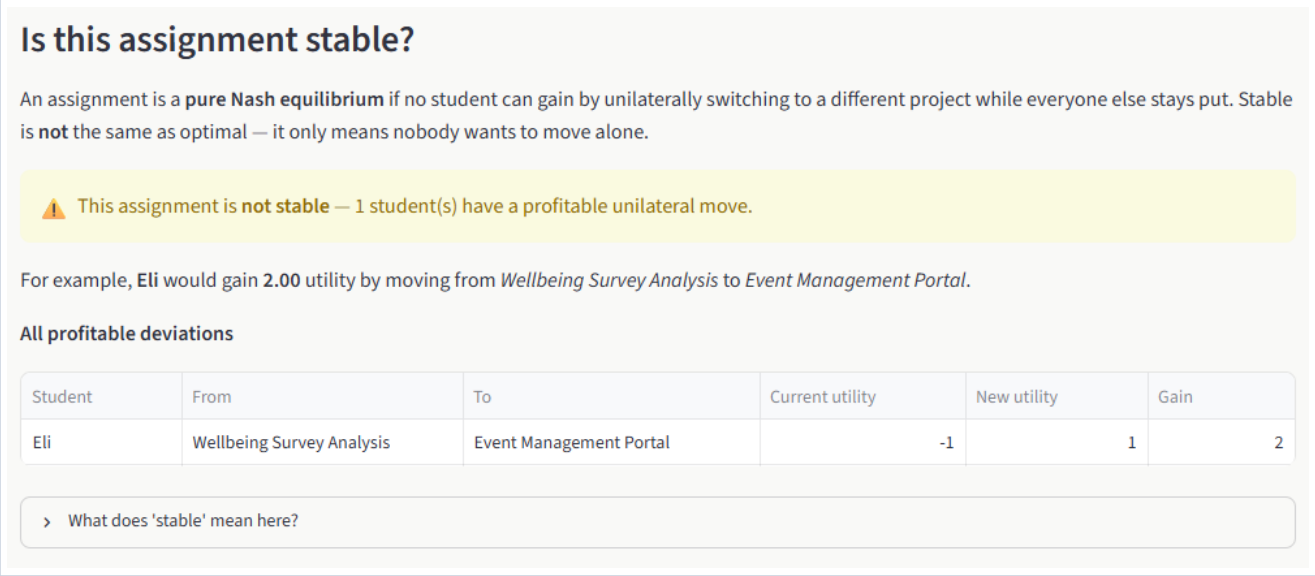


Figure 11. The welfare-29 assignment [0,0,1,0,1,1,2,2]: not stable, Eli gains 2. (file `screenshots/S8b_equilibrium_welfare_optimum_not_stable.png`*, region of the page; real engine; app code of commit* `c94f918`*; automated Chrome run.)*

Honest status of the browser evidence

These screenshots come from an **automated** Chrome run (Playwright) in which every on-screen metric and message was compared with the engine. A **manual** walkthrough by a team member on the presentation machine has **not** yet been done. Dataframe contents are drawn on a canvas and were checked by eye. Two cosmetic defects were observed and not fixed: an empty rounded bar above the title, and the Simulation chart's legend overlapping its title.

7. Testing and validation

Latest recorded run

`python -m pytest -q` → **158 passed, 0 failed, 0 skipped** in 15.69 s, on commit `34e2007`, clean working tree, fresh virtual environment, 2026-10-26 02:11:39 +0530. Source: `docs/results/pytest_output.txt`.
(Collection on the final commit `cdd6ec0` finds 158 tests; only evidence/PDF files changed after the recorded run.)

7.1 What each test group establishes, and what it does not

Test file	Tests	What it establishes	What it does NOT establish
`tests/test_game_engine.py`	101	Hand-solved fixtures (TINY, TRIO, COORD, REPEL), the exact-potential and welfare identities as property tests, tie handling, input validation, immutability, determinism, enumeration on small games, matching pennies has no pure equilibrium, dynamics endpoints lie in the enumerated set.	That the model is a good model of real teams; correctness of any code outside the engine.
`tests/test_integration.py`	26	The documented UI contract: return keys and types, JSON-serialisability, pandas/NumPy inputs, error behaviour, the CLI report, and that saved results in `docs/results/` are not stale (test I06).	That the UI uses the contract correctly (that is the UI tests' job).
`tests/test_ui_smoke.py`	26	The Streamlit app boots with four tabs and no exception, the Run dynamics button works, the real engine is used (not the mock), a broken-but-present engine is not misreported as missing, strict mode raises, the distinct-student count (8, not 16).	Table contents or chart pixels; it uses Streamlit's AppTest, not a real browser. Some tests check the **mock** engine against the tiny example, which proves the mock is faithful, not that the real engine is.
`tests/test_independent_crosscheck.py`	5	A separate computation from `docs/MODEL.md` reproduces 6,561 / 23 / 29 / PoS 1 / PoA 29/16, the potential identity on every profile and deviation, and agrees with the engine, including dynamics from all 6,561 starts.	Independence of authorship (same author as the engine); not a substitute for a second person's oracle.

7.2 Real engine versus mock engine

The mock in `app_helpers.py` was written so the frontend could be built before the engine existed. With the engine present it is never selected. `test_real_engine_loads_successfully_when_available` asserts `using_mock is False` and that `INTERFACE_VERSION == "1.0"`; the header of every screenshot shows "Real engine detected". A mock figure would carry a visible mock badge.

7.3 Known limitations and defects (stated honestly)

- **Pending verification:** the formal independent re-implementation and Excel oracle (tasks Q4, Q5, O-01..O-04) are not in the repository; the cross-check here is prepared by the engine's owner.
- **Pending:** a manual walkthrough on the presentation machine; the official title and faculty rules.
- **Cosmetic UI defects (open):** empty rounded bar above the page title (the header-card `<div>` cannot wrap Streamlit elements); legend overlaps the title of the Simulation chart.
- **Browser testing was automated:** Streamlit's canvas tables are not readable as text, so team compositions were checked by eye against the engine, not by script.
- **Streamlit floor:** `streamlit>=1.51` was determined by inspecting the 1.50.0 and 1.51.0 wheels (width parameter on `st.plotly_chart`); the full suite was run on 1.66.1 only.
- **Timings** are measured on one machine and are indicative only.
- **Synthetic data:** results illustrate the model, not real student behaviour.

8. Dependencies, installation and execution (Windows, verified)

These commands were run in a fresh virtual environment for this document. Use PowerShell from the repository root.

```
git clone https://github.com/AryanRajguru22/STRATOS.git
cd STRATOS
python -m venv .venv
.venv\Scripts\activate
python -m pip install -r requirements.txt
streamlit run app.py          # opens http://localhost:8501
python -m pytest -q          # full test suite
```

Purpose	Command	Expected result
Run the app	`streamlit run app.py`	Browser opens; header shows green "Real engine detected: game_engine.py and sample_data.py loaded."
Run all tests	`python -m pytest -q`	`158 passed` (about 20 s)
Engine report (default scenario)	`python game_engine.py --report`	JSON: 6,561 profiles, 23 equilibria, optimum welfare 29, PoS 1.0, PoA 1.8125
Independent cross-check	`python scripts/independent_crosscheck.py --compare`	Ends with `comparison with game_engine: IDENTICAL` (about 4 s)
Regenerate saved evidence	`python scripts/generate_results.py`	Rewrites `docs/results/*`; run it only on a clean, committed tree so the provenance is true
Strict engine mode	`$env:STRATOS_REQUIRE_REAL_ENGINE="1"; streamlit run app.py`	Any engine problem becomes a hard error instead of a mock
Different port	`streamlit run app.py --server.port 8765`	Use if 8501 is busy
Health check	`curl http://localhost:8501/_stcore/health`	ok

Roles of the four packages

- **Streamlit** runs the web app and provides tabs, widgets, session state and the `AppTest` harness used in tests.
- **pandas** holds the tables that the UI displays (students, teams, utilities, deviations, the dynamics trace).
- **Plotly** draws the two heatmaps, the utility bar chart and the welfare/potential line chart. It was missing from `requirements.txt` until commit `859a798`; a fresh install without it fails with `ModuleNotFoundError: No module named 'plotly'`.
- **pytest** discovers and runs the tests (`pytest.ini` sets `pythonpath = .` and `testpaths = tests`).

If something goes wrong

Symptom	Likely cause	Fix
`ModuleNotFoundError: plotly` (or streamlit)	Dependencies not installed in the active environment	Activate `.venv`, then `python -m pip install -r requirements.txt`
`.venv\Scripts\activate` is blocked	PowerShell execution policy	`Set-ExecutionPolicy -Scope Process Bypass`, then activate again
`TypeError` mentioning `width` on a button/chart	Streamlit older than 1.51	`python -m pip install -U "streamlit>=1.51"`
Red box "found but failed to import"	`game_engine.py` / `sample_data.py` has a bug or was damaged	Run `python game_engine.py --report` to see the real error; restore the file from git
Page does not load	Port busy or server stopped	Close the other server or use `--server.port`
A number differs from this handbook	Different preset, gamma, assignment, seed or update order in the sidebar	Compare with `python game_engine.py --report`; use the Default scenario, dynamics seed 7, max sweeps 50

9. Likely viva questions and model answers

Answers are written to be **spoken**: lead with the one-sentence answer, then add the supporting detail only if asked. Tags show the topic. Items marked **[PENDING]** or **Answer honestly** need a decision from the team before the viva.

9.1 Project motivation and scope

[basic] Q. What problem does STRATOS address?

Forming project teams when students have preferences over projects, differ in how well they work together, and dislike crowded teams. Because each student's outcome depends on others' choices, we treat it as a game and ask which assignments are stable, how they compare with the best assignment, and whether repeated switching settles down.

[basic] Q. Why not just choose the assignment with the highest total welfare?

Because it might not be stable. In our default scenario the maximum welfare is 29, attained by two assignments, and one of them is not an equilibrium: Eli gains 2 by moving. A planner can propose it, but a student with a profitable move has a reason to defect. Game Theory lets us ask about stability, not only about total welfare.

[basic] Q. Is this real student data?

No. Every student, preference and weight is fictional and synthetic, chosen so strategic behaviour is visible in a small demonstration. The app and README say so. The results illustrate how the model behaves, not what real students would do.

[scope] Q. Is STRATOS a system a university could deploy to assign teams?

No. It is a demonstration and analysis tool for a course project. It has no capacity limits, no real data and no authentication, and it analyses only unilateral deviations.

[scope] Q. What is deliberately out of scope?

Coalition-proof equilibria, hard team-size capacities, incomplete information, mixed strategies, real data and large-scale heuristics. Exhaustive enumeration is capped at 100,000 assignments.

Q. What is the official title of your project?

[PENDING] Do not guess. Say: "Our topic is strategic behaviour in university course / project team formation; we are confirming the exact title and group row against the official sheet." Ayush owns this check (task Q1). Update this answer once it is done.

[arch] Q. Why did you build the interface in Streamlit?

It lets us write the UI in the same language as the engine, gives interactive widgets and charts with little code, and starts with a single command (`streamlit run app.py`). The engine is independent of it, so the UI could be replaced without touching the mathematics.

9.2 Game Theory fundamentals

[GT] Q. What are the three ingredients of a game, and what are they here?

Players: the 8 students. Strategies: each student's choice of one of 3 projects. Payoffs: the utility function u_i(s) = preference + compatibility with teammates − crowding cost.

[GT] Q. Is this a simultaneous or sequential game?

Static and simultaneous with complete information: everyone chooses at once and knows all parameters. Best-response dynamics is an adjustment process we simulate to see where students might settle; it is not a sequential game.

[GT] Q. What is a best response?

The project that maximises a student's utility given everyone else's current choices. Ours keeps the current project if it is already a maximiser, otherwise takes the lowest-index maximiser.

[GT] Q. Define a Nash equilibrium. Is yours strict or weak?

A profile where nobody can gain by deviating alone. Ours is the weak form: ties do not count as profitable deviations (gain must exceed 10^{-9}), so an equilibrium may leave a student indifferent between projects.

[GT] Q. Does every game have a pure Nash equilibrium?

No. Matching pennies has none (our generic checker confirms it returns an empty list). Every finite game has a mixed equilibrium (Nash's theorem), but for our model the potential-game argument guarantees a **pure** one, so we do not need mixed strategies.

[GT] Q. Is there a dominant strategy for a student?

Not in general. A student's best project depends on who else is where and how crowded each team is, so the best choice changes with the others' choices. That is why equilibrium, not dominance, is the right concept.

[GT] Q. What is the difference between Pareto optimality and welfare maximisation?

Welfare maximisation picks the assignment with the largest sum of utilities. Pareto optimality only says no other assignment makes everyone at least as well off and someone strictly better off. A welfare maximiser is always Pareto-optimal, but a Pareto-optimal assignment need not maximise welfare. 20 of our 23 equilibria are Pareto-optimal.

[GT] Q. Give an everyday analogy for 'stable but not optimal'.

The prisoner's dilemma: each player's best reply is to defect, so (defect, defect) is an equilibrium, yet both would be better off cooperating. In our game the worst equilibrium has welfare 16 while the optimum is 29.

9.3 Mathematical model and equations

[model] Q. Write the utility function and explain each term.

$u_i(s) = p(i, s_i) + \Sigma$ over teammates j of $w(i,j) - \gamma(k_i - 1)^2$. The first term is how much i likes their project, the second is the summed compatibility with the students who share it (positive = synergy, negative = friction), and the third is a crowding cost that grows quadratically with team size k_i (counting i).

[numeric] Q. Compute the crowding cost for a team of 5 when $\gamma = 2$.

$c(5) = 2 \cdot (5-1)^2 = 2 \cdot 16 = 32$ for each member, so the team pays $5 \cdot 32 = 160$ in total.

[numeric] Q. Three students share a project. Each has preference 4 for it, every pair has $w = 1$, and $\gamma = 1$. What is each utility and the welfare?

$k = 3$, so $c(3) = 1 \cdot 2^2 = 4$. Each has two teammates: $u = 4 + (1 + 1) - 4 = 2$. Welfare $= 3 \cdot 2 = 6$.

[model] Q. Why must w be symmetric?

Each pair's compatibility enters the potential once, and enters both members' utilities. If $w(i,j) \neq w(j,i)$, a move by i would change the potential by something different from i's utility change, and the exact-potential property fails. The engine rejects asymmetric w.

[model] Q. What does a negative w mean? What does $w = 0$ mean?

Negative: two students work badly together, so sharing a project reduces utility. Zero: no effect. With our skill-based data, two students with identical skill sets get $w = 0 - 1 = -1$.

[model] Q. Why is welfare $W(s) = \Sigma p + 2\Sigma w - \Sigma\, n \cdot c(n)$, with a factor 2?

Summing every student's utility counts each compatible pair twice (once for each member), so the pair term is doubled, while the potential counts it once. The crowding terms add up to $n(t) \cdot c(n(t))$ per project.

[model] Q. What is γ and what happens at $\gamma = 0$?

γ scales the crowding cost. At $\gamma = 0$ crowding disappears, so compatibility and preference alone decide, and large teams are not penalised. Larger γ pushes teams to be smaller and more balanced.

[model] Q. Do utilities have units? Is adding them up meaningful?

They are on an arbitrary common scale (preferences are 0–10). Adding them assumes utilities are comparable across students, which is a modelling assumption, not a fact; we flag this as a limitation of welfare as a measure.

[model] Q. What does "ties" mean for the equilibrium definition?

If a student is exactly indifferent, moving is not a profitable deviation, so the profile can still be an equilibrium. We use the weak definition because it is the standard one and keeps the best-response rule deterministic.

9.4 Nash equilibrium and potential games

[potential] Q. What is a potential game?

A game with a function Φ over strategy profiles such that any player's change in payoff from a unilateral deviation equals the change in Φ. If it exists, local maxima of Φ are pure equilibria.

[potential] Q. State your potential function and prove the key property.

Φ(s) = Σ p(i,s_i) + Σ_{same-team pairs} w(i,j) − Σ_t Σ_{k=1}^{n(t)} c(k). When i moves from team a to team t, only the terms involving i and these two teams change, and the change equals [p(i,t) + Σ w on t − c(n(t)+1)] − [p(i,a) + Σ w on a − c(n(a))], which is exactly i's utility change.

[potential] Q. Why does the potential guarantee that an equilibrium exists?

There are finitely many assignments, so Φ has a maximum. From a maximiser no deviation can raise Φ, hence none can raise the deviator's utility, so it is a pure equilibrium.

[potential] Q. Why does best-response dynamics converge?

Every strictly profitable move raises Φ by exactly the mover's gain, so Φ strictly increases, assignments cannot repeat, and there are finitely many. With integer data each move raises Φ by at least 1, bounding the number of moves by Φ_max − Φ_min.

[potential] Q. Does converging mean reaching the best equilibrium?

No. From the same start, round-robin ended at welfare 29 and random order (seed 7) at 23; both are equilibria. The dynamics finds an equilibrium, not the best one.

[potential] Q. Is every equilibrium a maximiser of Φ?

No. Φ's maximum is 49, but equilibria have other Φ values (the table of 23 equilibria shows several). Φ-maximisers are equilibria; the converse fails.

[potential] Q. Could your proof fail for some data?

It requires symmetric w and a crowding cost depending only on team size. Asymmetric compatibility or a non-pairwise team bonus breaks it. That is why the engine validates symmetry and why the code never relies on the theorem: it checks the equilibrium condition directly.

9.5 Algorithms and implementation

[algo] Q. How do you check whether an assignment is a Nash equilibrium?

For every student and every other project, build the deviated assignment, compute that student's utility, and compare it with the current one. If any gain exceeds $\varepsilon = 10^{-9}$ the assignment is not stable. Cost $n(m-1) = 16$ utility evaluations for the default scenario.

[algo] Q. How does exhaustive enumeration work and what does it cost?

It generates all m^n assignments, computes utilities, welfare and potential for each, keeps those with no profitable deviation, and finds the welfare and potential maximisers; Pareto-optimality compares each equilibrium with every other assignment. It is exponential in n; the default takes about 0.1207 s, and it refuses more than 100,000 assignments.

[algo] Q. Why is the result deterministic?

Visiting order is fixed (round_robin) or a seeded permutation (random); ties are broken by the lowest project index; there is no hidden randomness. The same inputs and seed give identical output.

[algo] Q. What does the engine do with invalid input?

Every public function validates its input and raises `StratosError` (or `ScenarioError`/`ProfileError`) with a message naming the offending field and index, e.g. an asymmetric w. Booleans, strings, NaN and infinities are rejected. The UI catches the error and shows it instead of crashing.

[algo] Q. Does the engine modify its inputs?

No. It compiles the scenario into an internal copy; tests check that inputs are unchanged. Outputs are plain lists, dicts and numbers so `json.dumps` always works.

[algo] Q. Why does `best_response_dynamics` return `is_nash_final` and the UI re-checks anyway?

`converged` only says one sweep had no move. Stability is a separate claim, so the engine recomputes it from scratch and the UI does a fresh `is_nash` call as well. It is cheap and removes any trust in a flag.

[algo] Q. What is `rounds` in the dynamics result?

The number of sweeps performed, including the final sweep in which nobody moved. For the demo start under round-robin: 3 sweeps and 4 moves.

[algo] Q. How would the method scale to 100 students?

Dynamics scales fine: a sweep costs $O(n^2 m)$. Enumeration does not (m^n); you would drop it and rely on dynamics from many starts, or on integer programming/heuristics to look for the welfare optimum.

9.6 Architecture and frontend

[arch] Q. Why is the engine separate from the UI?

So it can be tested and run without a browser, reused by the CLI report, and verified independently. The engine imports neither Streamlit, NumPy nor pandas, and the interface between them is a documented contract (`docs/INTERFACE.md`).

[arch] Q. How does a click on a selectbox end up as a number on screen?

The selection is stored in `st.session_state.profile` (a list of project indices). On rerun the app validates it, calls `utilities`, `teams`, `social_welfare` and `potential`, converts the results into DataFrames and Plotly figures through `app_helpers`, and Streamlit redraws.

[arch] Q. What is the mock engine and why does it exist?

A contract-mirroring stand-in in `app_helpers.py`, written so the frontend could be developed before the engine existed. With the real engine present it is never used; if the engine is missing the app labels everything as mock, and if it is present but broken the app shows a red error. `STRATOS_REQUIRE_REAL_ENGINE=1` makes any fallback an error.

[arch] Q. How do you know the screen shows the engine's numbers?

Tests assert that the real engine is loaded, and the Chrome walkthrough compared the on-screen metrics (welfare, potential, sweeps, moves, status, messages) with values computed by the engine for 17 states.

[arch] Q. What happens if a user supplies bad input?

The UI only offers valid choices, and any `StratosError` from the engine is shown with `st.error`. The app does not crash. The Custom preset's sliders are bounded.

[arch] Q. Why does the Simulation tab not change the Assignment tab's assignment?

Dynamics starts from the current assignment and keeps its own result in `last_dynamics_result`. This lets you run different update orders from the same start, which is exactly what the demo does.

[arch] Q. Why did you replace `use_container_width` and what did that require?

Streamlit deprecated it in favour of `width="stretch"`. That parameter exists on `st.plotly_chart` only from Streamlit 1.51, so the declared minimum was raised from 1.30 to 1.51 (checked in the 1.50.0 and 1.51.0 packages).

9.7 Testing and independent validation

[test] Q. How do you know your engine is correct?

Several layers: hand-solved fixtures, property tests (potential and welfare identities on random games), tie and validation tests, enumeration checked against dynamics endpoints, matching pennies through a generic checker, a CLI report, and an independent recomputation that agrees exactly. Latest recorded run: 158 passed.

[test] Q. What does the independent cross-check do and what are its limits?

It re-derives utilities, welfare, potential, equilibria, optimum, Pareto flags, PoA/PoS and dynamics from MODEL.md with different code and exact fractions, without importing the engine for its own numbers, then diffs against the engine. Limit: it shares an author and a specification with the engine. A second person's re-implementation and an Excel oracle are planned and not yet in the repository.

[test] Q. What does test I06 do?

It fails if `sample_data.py` or the engine changes without regenerating `docs/results/`, so saved evidence can never silently go stale. A failure there means the check is working.

[test] Q. What does `AppTest` check, and what doesn't it?

It runs the Streamlit script headlessly and lets tests click widgets and inspect elements: no exceptions, four tabs, the dynamics button works, messages appear. It does not render pixels or canvas tables, so we added an automated Chrome run for screenshots. A manual walkthrough on the demo machine is still pending.

[test] Q. Which test would catch a wrong crowding formula?

The hand-solved fixtures (for example the tiny example's utilities [6, 2] at [0, 0], which depend on c(2) = 1) and the independent cross-check, which recomputes every utility with its own implementation of c(k).

[test] Q. Did the tests ever find a real defect?

Yes, in the course of the project: Plotly was imported but missing from `requirements.txt`, which broke a fresh install; and the Equilibrium tab counted deviation entries instead of students (showed 16 for 8 students). Both were fixed, the second with a regression test.

[test] Q. What could still be wrong that your tests would not catch?

A mistaken modelling assumption (the tests verify the code against the model, not the model against reality), synthetic data being unrepresentative, and any visual defect not covered by an assertion, such as the two cosmetic issues we know about.

9.8 Results and interpretation

[results] Q. Summarise your main numerical results.

For the default scenario: 6,561 assignments, 23 pure Nash equilibria, maximum welfare 29 (two optimal assignments, only one stable), price of stability 1.0, price of anarchy 1.8125 (= 29/16), and 20 of the 23 equilibria Pareto-optimal.

[results] Q. What does a price of stability of 1.0 mean?

The best equilibrium achieves the maximum possible welfare (29), so stability does not cost anything in the best case. Our enumeration found that the welfare-29 assignment [0,0,1,0,2,1,2,2] is itself an equilibrium.

[results] Q. What does the price of anarchy of 1.8125 mean?

The worst equilibrium has welfare 16 and 29/16 = 1.8125: selfish stable behaviour can leave the class with about 55% of the achievable welfare. It is a worst-case measure over equilibria.

[results] Q. Why are there 23 equilibria and not 1?

Many different assignments are mutually consistent: moving one student alone does not pay in any of them. Because the game has many local optima of Φ (and equilibria that are not Φ-maximisers), there is no unique stable outcome.

[results] Q. Why can the two update orders end differently?

Each order is a different path through the assignments; each path climbs Φ to some local maximum in the sense of unilateral moves, and there are 23 such endpoints. All 6,561 starts under round-robin end in one of them.

[results] Q. Why did the ratios become None at $\gamma = 3$?

Higher crowding makes some equilibria have non-positive welfare, so dividing the optimum by that welfare is meaningless. The engine returns null and reports the gaps instead (here 0 and 13).

[results] Q. If students are rational, why would they end at the worst equilibrium?

Rationality only gives local improvements. From a bad starting assignment, a sequence of individually sensible moves can end in a stable but poor assignment, because nobody can coordinate a joint move. That is exactly the cost of lacking coordination.

[results] Q. What would you recommend a planner do, based on the model?

Use the equilibrium analysis to find an assignment that is both high-welfare and stable (here [0,0,1,0,2,1,2,2], welfare 29), rather than only maximising welfare or letting students settle on their own. With synthetic data this is an illustration, not advice about real classes.

9.9 Limitations, trade-offs and future improvements

[limits] Q. What are the main limitations?

Synthetic data and assumed weights; complete information; only unilateral deviations; additive pairwise compatibility; no hard capacities; exhaustive search is exponential; the UI does not display PoA/PoS.

[limits] Q. What would you add with more time?

An Analysis tab showing equilibria and efficiency in the UI; coalition-proof analysis; hard capacities; real preference data; a heuristic search for large classes; and the independent oracle.

[limits] Q. Why unilateral deviations only?

It is the standard Nash notion and keeps the potential structure. Groups moving together require coalition concepts (strong equilibrium, coalition-proof equilibrium), which are harder to compute and may not exist.

[limits] Q. What is the trade-off between soft crowding and a hard cap?

Soft crowding keeps every deviation feasible and preserves the potential property, but cannot enforce a maximum size. A hard cap is more realistic but makes the feasible set depend on others' choices and can break existence results.

[limits] Q. How sensitive are results to the data?

Considerably: the number of equilibria and the PoA depend on preferences, compatibility and γ (at $\gamma = 3$ the ratios are undefined). We deliberately chose data to make strategic behaviour visible, and say so.

9.10 Individual contribution and teamwork

[team] Q. Who did what?

See section 10: the ownership map is built from the repository's git history and the team plan, and lists what is evidenced and what is pending. Each member must describe only their own part and say "I understand and can explain" for the rest.

[team] Q. Tell me about a problem you found and fixed.

Examples from the record: the missing Plotly dependency (found by a fresh-install test), the deviation count that showed 16 for 8 students (found in review, fixed with a regression test), the silent mock fallback (now a loud error), and a stale Streamlit minimum version.

[team] Q. How did you make sure the pieces fit together?

A written interface contract (`docs/INTERFACE.md`), integration tests against it, a results generator with git provenance, and fresh-environment installs.

Q. How much of the work did the tools do?

Answer honestly. Parts of the code, tests, documentation and these PDFs were prepared with AI assistance (Claude) under the team's direction and were then run and checked. Each member should be able to explain the part they present without notes. If asked, say so plainly.

[team] Q. What did you personally not do, and who did?

Be ready to say, for example: the engine and model are Aryan's, the app is Harsh's, verification and documentation tasks are Ayush's; whoever presents the rest explains it but does not claim to have built it.

10. Four-member contribution and ownership map

This map is built only from what the repository and the team plan show. **Do not claim anything that is not supported here or that you did not do yourself.** Commits are listed by the author name recorded in git.

Member	Role in the team plan	Evidence in the repository	Be prepared to defend
Aryan Rajguru	Game theory engine, mathematical model, technical integration and sign-off; owns `game_engine.py`, `sample_data.py`, `docs/MODEL.md`, `docs/INTERFACE.md`, engine and integration tests, `docs/results/`, `requirements.txt`	`cdd6ec0` 2026-10-26 Extend real-browser walkthrough with the short-demo path... `7e67155` 2026-10-26 Regenerate docs/results from the final tree `34e2007` 2026-10-26 Add real-browser screenshots and walkthrough log `8e396da` 2026-10-26 Add README.md and requirements-docs.txt `c25238b` 2026-10-26 Add independent cross-check of the model results `cd9fdc7` 2026-10-26 Raise Streamlit floor to 1.51; refresh install docs after... `859a798` 2026-10-26 Add Plotly to project requirements `49645b7` 2026-10-25 Add reproducible STRATOS engine results `52f0e6a` 2026-10-25 Implement STRATOS game theory engine	The whole model: utility, potential proof, equilibrium and dynamics, enumeration and PoA, test strategy, the results and their limits
Harsh Kumar Singh	Frontend: Streamlit app, UI integration, UI smoke tests; frontend README draft	`c94f918` 2026-10-26 Fix frontend deviation count and engine safeguards `8de4449` 2026-10-26 Merge remote-tracking branch 'origin/feat/ui' `c3d19ff` 2026-10-25 docs: add frontend README handoff draft for Ayush `3632842` 2026-10-25 feat(ui): surface the engine's synthetic-data disclaimer `17cc76a` 2026-10-25 Merge feat/engine into feat/ui: integrate real... `fa0ebbe` 2026-10-25 feat(ui): improve Streamlit frontend and integration readiness	The four tabs, every control, session state, engine loading and the mock fallback, UI tests, the live demonstration
Ayush	Per the team plan: independent verification (Q4, Q5, O-01..O-04), README, test plan/log, report, slides, demo script, viva file, screenshots	**No commits by Ayush were found in the repository** when this document was built, and none of the plan's Ayush deliverables (`tests/oracle/`, report, slides, `docs/TEST_LOG.md`) are present. This is not a judgement of the work done off-repository; it is what can be evidenced.	Whatever he has actually done. He should explain the testing and verification evidence in this handbook, and **claim only his own work**
Kashish	Fourth team member. Presents and explains selected parts of the completed product (problem, concepts, results, limitations).	No implementation, testing or documentation is attributed to Kashish, and none should be claimed.	Problem and motivation, what each tab is for, utility / Nash equilibrium / best-response dynamics in plain words, the meaning of the results, limitations and future work. Must also understand the whole system.

Work prepared for the final release (not attributable to a single team member)

Prepared by Aryan with AI assistance and committed on branch `release/final-product`: the independent cross-check and its tests, `README.md`, `requirements-docs.txt`, the Streamlit minimum-version fix, the real-browser screenshots with their index, the regenerated `docs/results/`, and these two PDFs. Ayush should review the README and the verification text because they fall in his areas.

Who can be asked about what

Topic	Primary	Everyone must still be able to say
Utility, potential, equilibrium, dynamics, enumeration, results	Aryan	The utility equation, what Nash equilibrium means, why stable ≠ optimal, the headline numbers
Tabs, controls, state, demo, UI tests	Harsh	What each tab shows and how to read it
Testing strategy, verification evidence, documentation, conclusions	Ayush (to confirm what he did)	What the tests establish and what is still pending
Motivation, concepts in plain language, interpretation, limitations	Kashish	How the pieces fit together (architecture)